UNIVERSIDAD AUTONOMA GABRIEL RENE MORENO FACULTAD DE INGENIERÍA EN CIENCIAS DE LA COMPUTACIÓN Y TELECOMUNICACIONES



PARCIAL 1: *SIMULACIÓN DE UNA CIUDAD INTERACTIVA EN OPENGL*

ESTUDIANTE: *ISEVICH ALCON MANUEL ODAIR*

NRO REGISTRO: *223049107*

MATERIA: ***PROGRAMACIÓN GRÁFICA***

SEMESTRE II/2026

1. ¿De qué trata el proyecto?

El proyecto consiste en la creación interactiva de una **ciudad tridimensional en tiempo real** donde el usuario conduce un vehículo explorando calles, avenidas y manzanas urbanas.
A diferencia de usar motores ya armados (como Unity o Unreal), todo el mundo visual —las calles, los edificios, la iluminación nocturna, el vehículo, las colisiones y el minimapa— fue **construido desde cero con programación gráfica pura**, comunicándose directamente con la tarjeta de video mediante instrucciones de OpenGL y shaders GLSL.
El objetivo fue transformar un prototipo básico inicial de 7x7 celdas en un entorno mucho más amplio de **11x11 celdas**, dotándolo de vida urbana, física de conducción suave, iluminación dinámica (día/noche), señalización vial y un panel de control interactivo.

2. Cronología del Desarrollo: Evolución del Proyecto en 4 Etapas

El proyecto se estructuró de manera progresiva a través del tiempo, dividiendo el trabajo en cuatro clases de Java que heredan una de la otra:

- **Etapa 1: ciudad.java** - Cimientos de la Ciudad
- **Etapa 2: auto.java** - Movimiento, Física y Cámara (Añade física, cámara y colisiones)
- **Etapa 3: iluminacion.java** - Iluminación y Atmósfera (Añade shaders, ciclo día/noche y farolas)
- **Etapa 4: decoracion.java** - Decoración, Minimapa y Entregas (Versión Final Completa)

Etapa 1: Cimientos de la Ciudad (ciudad.java)

Objetivo: Crear la ventana, el escenario urbano ampliado y la estructura de calles.

1. **Ampliación del Escenario (11×11):** Se diseñó una cuadrícula de 11x11 celdas conectadas por una red de calles sin callejones cerrados. La ciudad mide 110 metros de lado a lado.
2. **Distribución Urbana Equilibrada:** Se distribuyeron 19 manzanas de edificios con alturas y colores variados para darle silueta de ciudad moderna, y 6 parques verdes, cumpliendo y superando lo solicitado en la guía práctica.
3. **Geometría Base:** Para optimizar el rendimiento, todos los elementos se generan a partir de una figura básica en memoria: un cubo unitario. Modificando su tamaño, posición y color, se dibujan aceras, calzadas y rascacielos.
4. **Marcas Viales en las Calles:** Se añadieron líneas amarillas discontinuas a lo largo de las calzadas que respetan y dejan libres las intersecciones de tránsito.

Etapa 2: Movimiento, Física y Cámara (auto.java)

Objetivo: Permitir al usuario conducir el vehículo sintiendo peso y control natural.

1. **Movimiento Fluido Independiente de los FPS (Delta Time):** Se programó el cálculo de movimiento multiplicándolo por el tiempo transcurrido entre fotogramas (deltaTime). Gracias a esto, el vehículo se desplaza a la misma velocidad en cualquier computadora, sin importar si corre a 30 o a 144 cuadros por segundo.
2. **Comportamiento del Vehículo:** El auto cuenta con aceleración progresiva,

desaceleración natural por fricción, marcha atrás y freno de mano con derrape suave. Además, el auto solo gira cuando está en movimiento, simulando la dirección de un automóvil real.
3. **Detección de Colisiones:** El vehículo no puede atravesar las aceras ni salirse del perímetro de la ciudad. Si colisiona con el cordón de una vereda, su avance se detiene de forma inmediata.
4. **Sistema de Cámaras:**
    - **Cámara en 3ra Persona:** Sigue al vehículo desde atrás y rota de forma solidaria cuando el conductor gira.
    - **Cámara Aérea Orbital (Tecla C):** Se eleva sobre el centro de la ciudad para contemplar toda la urbe desde las alturas.

Etapa 3: Iluminación y Atmósfera (iluminacion.java)

Objetivo: Dotar a la ciudad de realismo lumínico mediante shaders en la tarjeta gráfica.

1. **Ciclo de Día y Noche (Tecla N):**
    - **Día:** La ciudad recibe luz solar cálida y un ambiente luminoso y claro.
    - **Noche:** La escena pasa a un tono nocturno azulado oscuro, donde la visibilidad depende de las luces artificiales.
2. **Alumbrado Público (9 Farolas Distribuidas):** Se colocaron 9 farolas estratégicas en esquinas y avenidas. La luz no solo ilumina la bombilla, sino que baña el asfalto y las fachadas cercanas, disipándose suavemente a medida que los objetos se alejan.
3. **Faros Delanteros del Automóvil (Tecla F):** El vehículo incorpora dos focos frontales que apuntan hacia donde mira el coche, proyectando un cono de luz sobre la calzada para permitir la conducción segura durante la noche.
4. **Superficies Emisivas:** Las bombillas de las farolas y las luces del auto brillan con luz propia, manteniéndose visibles incluso en la oscuridad total.

Etapa 4: Decoración, Minimapa y Entregas (decoracion.java)

Objetivo: Finalizar la simulación urbana con detalles visuales, navegación y desafíos.

1. **Detalles y Vida Urbana:**
    - **Parques:** Cuentan con árboles 3D de copa verde y bancos de descanso de madera.
    - **Ventanas de Edificios:** Durante la noche, múltiples ventanas de los edificios se encienden simulando departamentos habitados.
    - **Pasos Peatonales:** Cruces de cebra con franjas blancas continuas que unen una acera con la de enfrente a través de la calle.
    - **Semáforos Temporizados:** Ubicados en las esquinas, alternan automáticamente entre Verde (5 s), Amarillo (2 s) y Rojo (5 s).
2. **Minimapa en la Esquina Superior (Tecla M):** Una vista satelital permanente que muestra en miniatura toda la ciudad, la posición y rumbo del auto (marcador cian con punta blanca) y la ubicación del próximo destino. El minimapa se orienta siempre con el Norte hacia arriba y no borra la vista principal.
3. **Sistema de Misiones de Entrega:** El jugador tiene como objetivo recorrer la ciudad visitando 3 paradas consecutivas señalizadas con plataformas y balizas giratorias en el suelo.
4. **Reinicio Rápido (Tecla R):** Permite reiniciar la posición del vehículo y el circuito de

entregas en cualquier momento.

Etapa 5 (Mejora Opcional): Señalización y Panel en Pantalla

Objetivo: Cumplir con los puntos de bonificación opcional del proyecto (+10 pts).

1. **Señalización Vial Urbana:** Grandes pórticos elevados sobre las avenidas con carteles reflectantes que indican sectores de la ciudad y postes con señales de límite de velocidad.
2. **Panel de Instrumentos (HUD) en la Esquina Inferior:** Tacómetro gráfico de velocidad, testigos de luces y contador de entregas completadas.
3. **Menú de Pausa y Ayuda de Controles (Tecla P):** Al presionar P, el auto se congela, la pantalla se oscurece y aparece una ventana con la lista clara de todos los controles disponibles.

3. Funciones Principales Implementadas (Estructura de Código)

A continuación se resumen los métodos fundamentales desarrollados en cada clase del proyecto, omitiendo funciones auxiliares menores para mantener la claridad y concisión del informe:

ciudad.java (Entorno, Malla Urbana y Geometría)

- **escena():** Recorre la matriz de 11 × 11 celdas para transformar los datos numéricos en asfalto, aceras elevadas, manzanas de edificios y parques.
- **dibujarMarcasCalle(fila, columna, x, z):** Dibuja las líneas amarillas discontinuas de las calzadas respetando los cruces, adaptando dinámicamente su grosor cuando se activa el minimapa para evitar pérdidas de rasterizado.
- **caja() / cajaGirada():** Aplica las transformaciones de traslación, escala y rotación enviando las propiedades al shader para renderizar cualquier volumen con cubos.
- **centro(indice):** Función matemática de conversión que transforma índices de matriz a coordenadas continuas del mundo 3D.

auto.java (Física del Vehículo y Cámaras)

- **actualizar(deltaTime):** Modela la aceleración progresiva, fricción natural, frenado de mano y cálculo de movimiento independiente de la tasa de refresco (FPS).
- **puedeCircular(x, z):** Algoritmo de detección de colisiones que restringe el paso del vehículo impidiendo invadir aceras elevadas o abandonar el perímetro de la ciudad.
- **configurarCamara():** Calcula la posición y ángulo visual del usuario, alternando entre tercera persona solidaria al coche y vista orbital aérea.
- **auto():** Ensambla procedimentalmente la carrocería, cabina, cuatro ruedas y faros del vehículo sobre sus coordenadas actuales.

iluminacion.java (Cálculo de Luz en GPU y Ciclo Día/Noche)

- **prepararLuces():** Pasa los uniforms de iluminación a la GPU (estado día/noche,

ubicación de las 9 farolas y orientación del vehículo).
- **fragmentShader():** Shader GLSL en GPU que calcula la iluminación difusa de Lambert y la fórmula de atenuación cuadrática por distancia.
- **escena():** Renderiza las farolas públicas asignando propiedad emisiva a las bombillas para que brillen en la oscuridad.

decoracion.java (Vida Urbana, Navegación y HUD)

- **dibujarMinimapa():** Ejecuta un segundo pase de renderizado con recorte (glScissor) y proyección cenital ortográfica para mantener el Norte siempre arriba sin alterar la vista principal.
- **decorarCiudad():** Distribuye los árboles 3D, bancos en parques y ventanas iluminadas en las fachadas durante la noche.
- **dibujarPasoPeatonal():** Genera pasos de cebra con franjas continuas que conectan el cordón de una acera con la de enfrente.
- **dibujarSemaforo():** Controla la animación lumínica temporal del semáforo alternando entre fases de verde, amarillo y rojo.
- **dibujarDestino():** Posiciona la plataforma interactiva y la baliza oscilante de las misiones de entrega.
- **dibujarHUD():** Superpone en 2D el tacómetro de velocidad, los testigos de luces y el menú modal de pausa y controles (P).

4. Guía de Controles

Tecla	Función / Acción
W / Flecha Arriba	Acelerar hacia adelante
S / Flecha Abajo	Frenar y dar marcha atrás (reversa)
A / Flecha Izquierda	Girar el volante a la izquierda
D / Flecha Derecha	Girar el volante a la derecha
Espacio	Freno de mano
C	Cambiar cámara (Vista detrás del auto vs Vista aérea de la ciudad)
N	Alternar entre Modo Día y Modo Noche
F	Encender o apagar los faros del auto
M/K	Mostrar u ocultar el Minimapa de navegación y para ocultar el HUD
P	Pausar la simulación y abrir el Menú de Ayuda en pantalla

Tecla	Función / Acción
R	Reiniciar la partida (auto al inicio y entregas a cero)
ESC	Salir del programa

5. Cómo Compilar y Ejecutar el Proyecto

El proyecto está gestionado con Apache Maven, lo que garantiza que cualquier computadora con Java pueda compilarlo sin pasos complicados.

- **Requisitos:** Java JDK 21+ (probado en Java 24) y Apache Maven.
- **Compilar el código fuente:** mvn clean compile
- **Iniciar la aplicación:** mvn exec:java
- **Prueba rápida automática:** mvn exec:java "-Ddemo.frames=60"

Como en el pom.xml está definido como com.grafica.decoracion, solamente es necesario poner mvn compile exec:java como es indicado arriba

6. Limitaciones y Alcance del Trabajo

- **Geometría basada en cubos:** Para mantener el proyecto autónomo y ligero sin depender de archivos pesados externos (.obj o .fbx), toda la ciudad se modela matemáticamente a partir de cajas tridimensionales.
- **Iluminación directa sin sombras proyectadas:** Las luces iluminan las superficies que tienen de frente, pero los edificios no proyectan sombras negras largas sobre el suelo.
- **Semáforos de rol decorativo:** Los semáforos cambian de color de forma realista por tiempo, pero el juego no obliga físicamente al vehículo a detenerse ante una luz roja.

8. Módulo de Tráfico y Movimiento Autónomo de Vehículos

Se incorporó un sistema de tráfico dinámico con vehículos autónomos que circulan respetando la red vial, señales de tránsito y la presencia de otros automóviles:

- **Navegación Autónoma y Patrones de Giro:** Los vehículos detectan intersecciones y eligen giros aleatorios o trayectorias continuas manteniendo su carril.
- **Interacción con Semáforos:** Los autos reaccionan al estado de los semáforos, deteniéndose en luz roja y reanudando la marcha en verde.
- **Prevención de Colisiones y Distancia de Seguridad:** Cada unidad evalúa la proximidad de otros vehículos en su trayectoria para ajustar la velocidad o frenar, evitando colisiones.
- **Renderizado Eficiente y Variedad Estética:** Se optimizó el renderizado en OpenGL reutilizando la geometría de los modelos y variando colores y dimensiones para dar diversidad visual a la ciudad.

9. Link Repositorio git

https://github.com/Ale3n/ProgGrafica.git